

Bài toán 1. Hệ thống lấy hàng truck và drone hoạt động song song, có ràng buộc time window

Input: Hệ thống lấy hàng gồm: 1 tập gồm K truck giống nhau, D drone giống nhau cùng xuất phát từ Depot 0 đi gom hàng từ tập khách hàng C. Trong đó, tập khách hàng C được chia làm 2 loại:

- C1 – tập khách hàng bắt buộc phải phục vụ xe tải (do khối lượng hàng cần lấy từ khách quá lớn hoặc nằm ngoài khả năng bay của drone);
- C2- tập khách hàng có thể phục vụ bởi cả drone hoặc truck.

Constraints:

- **Xe tải** chỉ chở được tải trọng tối đa M_T
- **Drone** chỉ chở được tải trọng tối đa M_D và bán kính bay tối đa là L_D (bán kính bay tối đa được định nghĩa là số km tối đa mà drone có thể bay được trong hành trình từ khi xuất phát khỏi depot, đi lấy hàng, đến khi đem hàng về depot)
- **Khách:**
 - Mỗi gói hàng của khách hàng i có tải trọng: d_i
 - Thời gian đợi tối đa của một khách hàng là L_w (thời gian đợi của khách = thời điểm gói hàng được đem về đến kho hàng trừ đi thời điểm gói hàng được lấy từ khách hàng $\leq L_w$)
 - Khách hàng i yêu cầu được phục vụ trong khoảng thời gian $[e_i, l_i]$ tức là: nếu xe đến khách hàng i trước thời điểm e_i thì phải chờ đến e_i mới lấy được hàng, xe không được đến khách i sau thời điểm l_i
- **Hành trình:** Mỗi xe tải/drone được phép thực hiện nhiều hành trình, mỗi hành trình được định nghĩa là xuất phát rỗng từ depot, đi lấy hàng từ 1 hoặc nhiều khách hàng sao cho không vi phạm các ràng buộc đã mô tả ở trên, rồi đem hàng quay trở về depot.
- Thời gian sạc/thay pin drone tại depot, thời gian phục vụ khách hàng coi như = 0. Mỗi khi drone xuất phát từ depot, là đã sạc pin đầy, do đó có thể bay được tối đa L_D km.

Output:

Xây dựng hành trình của các Truck, Drone

Sao cho: Tối thiểu hoá thời gian hoàn thành nhiệm vụ. (Thời gian hoàn thành nhiệm vụ được định nghĩa là thời điểm toàn bộ hàng hóa đã được các xe đem hết về đến depot; Vì hệ thống bắt đầu hoạt động từ thời điểm 0, do đó, nếu xe cuối cùng về depot tại thời điểm 1000, thì thời gian hoàn thành nhiệm vụ chính là 1000).

Init solution

Sort customers by increasing deadline

for each customer i :

feasibleMoves $\leftarrow \emptyset$

penalizedMoves $\leftarrow \emptyset$

for each compatible vehicle v :

Generate all moves:

- every position in every existing trip of v
- a new trip at every position in v 's trip sequence

for each move a :

Temporarily insert i using a

Reschedule the affected trip and later trips

if a violates a hard constraint:

discard a

else if a creates no additional TW violation:

add a to feasibleMoves

else:

compute $\Delta TW(a)$

add a to penalizedMoves

if feasibleMoves $\neq \emptyset$:

choose the move with minimum resulting makespan

else:

choose the penalized move lexicographically by:

1. minimum increase in TW violation
2. minimum resulting makespan
3. minimum additional travel distance

permanently insert i using the selected move

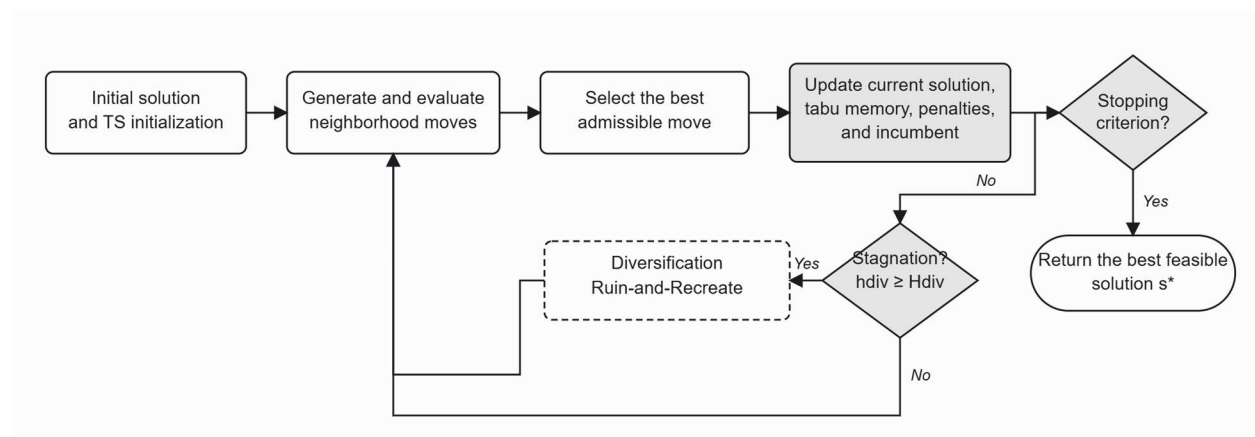
return the complete initial solution

Function compute $\Delta TW(a)$

1. $\text{penalty} \leftarrow 0$
2. for mỗi khách hàng j đã được chèn trong solution x của move a do
3. $\text{penalty} \leftarrow \text{penalty} + \max(0, t_j - l_j)$
4. end for
5. return penalty

TABU SEARCH

Trong quá trình tìm kiếm, nghiệm được phép tạm thời vi phạm tải trọng, thời gian bay drone trong mỗi trip, time window của khách và thời gian chờ hàng L_w . Nghiệm đầu ra bắt buộc thỏa tất cả ràng buộc.



1. Biểu diễn nghiệm

1.1. Ký hiệu đầu vào

Với mỗi khách hàng i :

- q_i : khối lượng hàng cần lấy.
- $[e_i, l_i]$: cửa sổ thời gian.
- $i \in C_1$: chỉ được phục vụ bằng truck.
- $i \in C_2$: có thể phục vụ bằng truck hoặc drone.

Thông số phương tiện:

- K : số truck.
- D : số drone.
- M_T, M_D : tải trọng tối đa.
- L_D : quãng đường tối đa của một chuyến drone.
- L_w : thời gian tối đa từ lúc lấy hàng đến lúc hàng về depot.
- τ_{ij}^v : thời gian đi từ i đến j bằng loại phương tiện v .
- d_{ij} : khoảng cách từ i đến j .

1.2. Cấu trúc một chuyến

Một chuyến của phương tiện v được biểu diễn:

$$\sigma_{vk} = [0, i_1, i_2, \dots, i_m, 0].$$

Trong đó:

- v : phương tiện thực hiện.
- k : vị trí của chuyến trong chuỗi chuyến của v .
- 0: depot.

Mỗi phương tiện có một danh sách chuyến có thứ tự:

$$R_v = (\sigma_{v1}, \sigma_{v2}, \dots, \sigma_{vm_v}).$$

Thứ tự các chuyến là một phần của nghiệm, vì thay đổi thứ tự trip sẽ làm thay đổi thời gian bắt đầu và thời gian đến của tất cả trip phía sau.

1.3. Cấu trúc dữ liệu

Customer:

id

demand

earliestTime

latestTime

droneEligible

Trip:

uid

vehicleId

customers[]

startTime

returnTime

load

travelDistance

arrivalTime[customerId]

waitingTime[customerId]

Vehicle:

id

type *// TRUCK hoặc DRONE*

trips[]

completionTime

Solution:

vehicles[]

makespan

normalizedMakespan

violationCapacity

violationRange

violationTimeWindow

violationWaiting

totalViolation

penalizedObjective

totalDistance

2. Kiểm tra tính tương thích tĩnh

Mặc dù tải trọng, tầm bay và thời gian chờ được phép vi phạm tạm thời ở cấp chuyển, không nên đưa một khách vào loại phương tiện mà khách đó không bao giờ có thể được phục vụ khả thi.

Ví dụ, khách i chỉ nên được xem là tương thích với drone nếu:

$$i \in C_2,$$

$$q_i \leq M_D,$$

$$d_{0i} + d_{i0} \leq L_D,$$

$$\tau_{i0}^D \leq L_w.$$

Điều này có nghĩa khách i có thể được drone phục vụ khả thi ít nhất trong chuyến đơn $[0, i, 0]$.

Pseudocode:

```
FUNCTION STATIC_COMPATIBLE(customer i, vehicle v):  
  
    IF v.type == DRONE:  
        IF i is not in C2:  
            RETURN false  
  
        IF demand[i] > droneCapacity:  
            RETURN false  
  
        IF distance[0][i] + distance[i][0] > droneRange:  
            RETURN false  
  
        IF travelTimeDrone[i][0] > maximumWaitingTime:  
            RETURN false  
  
    ELSE IF v.type == TRUCK:  
        IF demand[i] > truckCapacity:  
            RETURN false  
  
        IF travelTimeTruck[i][0] > maximumWaitingTime:  
            RETURN false  
  
    RETURN true
```


3. Tính lịch cho một phương tiện

Đây là hàm nền tảng quan trọng nhất. Tất cả các toán tử đều phải gọi hàm này sau khi thay đổi nghiệm.

3.1. Thời gian bắt đầu chuyến

Chuyến đầu tiên của phương tiện bắt đầu tại thời điểm 0:

$$t_{v,1}^{\text{start}} = 0.$$

Chuyến tiếp theo bắt đầu ngay sau khi chuyến trước trở về:

$$t_{v,k}^{\text{start}} = r_{v,k-1}.$$

3.2. Thời gian lấy hàng

Với trip:

$$\sigma = [0, i_1, i_2, \dots, i_m, 0],$$

thời gian lấy hàng tại khách đầu tiên:

$$a_{i_1} = \max \{ e_{i_1}, t_{\sigma}^{\text{start}} + \tau_{0i_1}^v \}.$$

Với các khách tiếp theo:

$$a_{i_h} = \max \{ e_{i_h}, a_{i_{h-1}} + \tau_{i_{h-1}i_h}^v \}.$$

Nếu $a_i > l_i$, phương tiện vẫn lấy hàng tại a_i , nhưng phát sinh vi phạm time window bằng $a_i - l_i$.

3.3. Thời gian trở về depot

$$r_{\sigma} = a_{i_m} + \tau_{i_m 0}^v.$$

Thời gian chờ của hàng tại khách $i \in \sigma$:

$$w_i = r_{\sigma} - a_i.$$

3.4. Pseudocode tính lại lịch

PROCEDURE RECOMPUTE_VEHICLE(solution s, vehicle v, firstAffectedTrip):

IF firstAffectedTrip == 0:

currentTime $\leftarrow$ 0

ELSE:

currentTime $\leftarrow$ v.trips[firstAffectedTrip - 1].returnTime

FOR k $\leftarrow$ firstAffectedTrip TO numberOfTrips(v) - 1:

trip $\leftarrow$ v.trips[k]

trip.startTime $\leftarrow$ currentTime

trip.load $\leftarrow$ 0

trip.travelDistance $\leftarrow$ 0

previousNode $\leftarrow$ depot

currentTime $\leftarrow$ trip.startTime

FOR each customer i in trip.customers:

currentTime $\leftarrow$ currentTime

+ travelTime(v.type, previousNode, i)

currentTime $\leftarrow$ max(currentTime, earliestTime[i])

trip.arrivalTime[i] $\leftarrow$ currentTime

trip.load $\leftarrow$ trip.load + demand[i]

**trip.travelDistance $\leftarrow$ trip.travelDistance
+ distance[previousNode][i]**

previousNode $\leftarrow$ i

**currentTime $\leftarrow$ currentTime
+ travelTime(v.type, previousNode, depot)**

**trip.travelDistance $\leftarrow$ trip.travelDistance
+ distance[previousNode][depot]**

trip.returnTime $\leftarrow$ currentTime

FOR each customer i in trip.customers:

**trip.waitingTime[i] $\leftarrow$ trip.returnTime
- trip.arrivalTime[i]**

IF numberOfTrips(v) == 0:

v.completionTime $\leftarrow$ 0

ELSE:

v.completionTime $\leftarrow$ last trip's returnTime

4. Đo mức vi phạm

Đặt:

$$[x]^+ = \max\{0, x\}.$$

Sử dụng một hằng số thời gian cố định:

$$H = \max \left\{ 1, C_{\max}(s_0), \max_{i \in C} l_i \right\}.$$

H được tính một lần từ nghiệm ban đầu và không thay đổi trong quá trình tìm kiếm.

4.1. Vi phạm tải trọng

Với trip σ :

$$Q(\sigma) = \sum_{i \in \sigma} q_i.$$

Tổng vi phạm tải trọng chuẩn hóa:

$$V_Q(s) = \frac{1}{n} \sum_{\sigma \in \mathcal{R}(s)} \frac{[Q(\sigma) - M_{v(\sigma)}]^+}{M_{v(\sigma)}}.$$

4.2. Vi phạm tầm bay drone

$$V_D(s) = \frac{1}{n} \sum_{\sigma \in \mathcal{R}_D(s)} \frac{[L(\sigma) - L_D]^+}{L_D}.$$

Trong đó $L(\sigma)$ là tổng quãng đường chuyển drone, tính cả đoạn từ depot và quay về depot.

4.3. Vi phạm time window

$$V_{TW}(s) = \frac{1}{n} \sum_{i \in C} \frac{[a_i - l_i]^+}{H}.$$

4.4. Vi phạm thời gian chờ hàng

$$V_W(s) = \frac{1}{n} \sum_{i \in C} \frac{[r_{\sigma(i)} - a_i - L_w]^+}{L_w}.$$

4.5. Tổng vi phạm

$$V_{\Sigma}(s) = V_Q(s) + V_D(s) + V_{TW}(s) + V_W(s).$$

Nghiệm khả thi khi:

$$V_{\Sigma}(s) \leq \varepsilon,$$

với $\varepsilon = 10^{-9}$ hoặc một sai số số học rất nhỏ.

4.6. Makespan chuẩn hóa

$$C_{\max}(s) = \max_v C_v(s),$$

trong đó $C_v(s)$ là thời gian phương tiện v hoàn thành chuyến cuối cùng.

Giá trị chuẩn hóa:

$$\bar{C}_{\max}(s) = \frac{C_{\max}(s)}{H}.$$

4.7. Hàm đánh giá có phạt

$$F_{\lambda}(s) = \bar{C}_{\max}(s) + \lambda_Q V_Q(s) + \lambda_D V_D(s) + \lambda_{TW} V_{TW}(s) + \lambda_W V_W(s).$$

Hàm này chỉ dùng để hướng dẫn quá trình tìm kiếm. Giá trị mục tiêu cuối cùng vẫn là makespan thực tế $C_{\max}$.

4.8. Pseudocode đánh giá nghiệm

PROCEDURE EVALUATE_SOLUTION(solution s , penaltyWeights λ):

FOR each vehicle v in s :

RECOMPUTE_VEHICLE(s , v , 0)

$s.\text{makespan} \leftarrow$ maximum completionTime among all vehicles

$s.\text{normalizedMakespan} \leftarrow s.\text{makespan} / H$

$V_Q \leftarrow 0$

$V_D \leftarrow 0$

$V_{TW} \leftarrow 0$

$V_W \leftarrow 0$

$\text{totalDistance} \leftarrow 0$

FOR each vehicle v :

$\text{capacity} \leftarrow \text{capacityOf}(v.\text{type})$

FOR each trip σ of v :

$VQ \leftarrow VQ$

$+ \max(0, \sigma.\text{load} - \text{capacity}) / \text{capacity}$

IF $v.\text{type} == \text{DRONE}$:

$VD \leftarrow VD$

$+ \max(0, \sigma.\text{travelDistance} - \text{droneRange})$

$/ \text{droneRange}$

$\text{totalDistance} \leftarrow \text{totalDistance} + \sigma.\text{travelDistance}$

FOR each customer i in σ :

$VTW \leftarrow VTW$

$+ \max(0, \sigma.\text{arrivalTime}[i] - \text{latestTime}[i])$

$/ H$

$VW \leftarrow VW$

$+ \max(0, \sigma.\text{waitingTime}[i] - \text{maximumWaitingTime})$

/ maximumWaitingTime

s.violationCapacity $\leftarrow$ VQ / n

s.violationRange $\leftarrow$ VD / n

s.violationTimeWindow $\leftarrow$ VTW / n

s.violationWaiting $\leftarrow$ VW / n

s.totalViolation $\leftarrow$ s.violationCapacity

+ s.violationRange

+ s.violationTimeWindow

+ s.violationWaiting

s.totalDistance $\leftarrow$ totalDistance

s.penalizedObjective $\leftarrow$ s.normalizedMakespan

+ $\lambda_Q \times$ s.violationCapacity

+ $\lambda_D \times$ s.violationRange

+ $\lambda_{TW} \times$ s.violationTimeWindow

+ $\lambda_W \times$ s.violationWaiting

5. Các nghiệm cần lưu

Thuật toán duy trì ba loại nghiệm.

5.1. Nghiệm hiện tại s

Đây là nghiệm mà Tabu Search đang đứng tại đó. s có thể khả thi hoặc không khả thi.

5.2. Nghiệm khả thi tốt nhất s^*

Chỉ cập nhật khi:

$$V_{\Sigma}(s) = 0$$

và:

$$C_{\max}(s) < C_{\max}(s^*).$$

Nếu chưa tìm được nghiệm khả thi thì:

```
bestFeasible ← null
```

5.3. Nghiệm không khả thi tốt nhất s^I

Hai nghiệm không khả thi được so sánh theo thứ tự:

$$(V_{\Sigma}(s), \bar{C}_{\max}(s), \text{Distance}(s)) .$$

Ưu tiên:

1. Tổng vi phạm nhỏ hơn.
2. Nếu bằng nhau, makespan nhỏ hơn.
3. Nếu vẫn bằng nhau, tổng quãng đường nhỏ hơn.

Pseudocode:

```
FUNCTION BETTER_INFEASIBLE(s1, s2):  
  
    IF s2 == null:  
        RETURN true  
  
    IF s1.totalViolation < s2.totalViolation - epsilon:  
        RETURN true  
  
    IF abs(s1.totalViolation - s2.totalViolation) <= epsilon:  
        IF s1.makespan < s2.makespan - epsilon:  
            RETURN true  
  
        IF abs(s1.makespan - s2.makespan) <= epsilon:  
            RETURN s1.totalDistance < s2.totalDistance  
  
    RETURN false
```

6. Các toán tử lân cận

Giữ năm toán tử trong Chương 3 và bổ sung **Trip-relocate**.

6.1. Relocate

Lấy một khách i khỏi vị trí hiện tại và chèn vào:

- Mọi vị trí trong mọi trip tương thích.
- Một trip mới tại mọi vị trí trong chuỗi trip của phương tiện tương thích.

Nếu trip nguồn trở thành rỗng thì xóa trip đó.

PROCEDURE GENERATE_RELOCATE(solution s , sourceCustomers):

moves $\leftarrow$ empty set

FOR each customer i in sourceCustomers:

```

sourceVehicle ← vehicle containing i
sourceTrip ← trip containing i
sourcePosition ← position of i
FOR each physical vehicle v:
    IF not STATIC_COMPATIBLE(i, v):
        CONTINUE
    FOR each existing trip t of v:
        FOR insertionPosition ← 0 TO size(t.customers):
            IF this is the original position:
                CONTINUE
            move ← Relocate(
                customer = i,
                source = current location,
                destination = (v, t, insertionPosition)
            )
            ADD move to moves
        FOR newTripPosition ← 0 TO numberOfTrips(v):
            move ← Relocate i into new trip [0, i, 0]
                inserted at newTripPosition of v
            ADD move to moves
RETURN moves

```

6.2. Or-opt(2)

Di chuyển một block gồm hai khách liên tiếp:

$$[i_h, i_{h+1}]$$

sang vị trí khác và giữ nguyên thứ tự block.

Cách sinh giống Relocate, nhưng đối tượng được di chuyển là một block hai khách.

```
PROCEDURE GENERATE_OR_OPT_2(solution s, selectedTrips):  
  
    moves ← empty set  
  
    FOR each selected trip sourceTrip:  
  
        FOR p ← 0 TO size(sourceTrip.customers) - 2:  
  
            block ← [  
                sourceTrip.customers[p],  
                sourceTrip.customers[p + 1]  
            ]  
  
            FOR each physical vehicle v:  
  
                IF any customer in block is incompatible with v:  
                    CONTINUE  
  
                FOR each existing trip t of v:  
                    FOR insertionPosition ← 0 TO size(t.customers):  
                        ADD corresponding Or-opt move  
  
                FOR newTripPosition ← 0 TO numberOfTrips(v):  
                    ADD move creating a new trip containing block  
  
    RETURN moves
```



6.3. Swap

Hoán đổi hai khách bất kỳ.

```
PROCEDURE GENERATE_SWAP(solution s, selectedCustomers):  
  
    moves ← empty set  
  
    FOR each unordered pair of customers (i, j):  
  
        vehicleI ← vehicle currently serving i  
        vehicleJ ← vehicle currently serving j  
  
        IF not STATIC_COMPATIBLE(i, vehicleJ):  
            CONTINUE  
  
        IF not STATIC_COMPATIBLE(j, vehicleI):  
            CONTINUE  
  
        IF swapping i and j changes nothing:  
            CONTINUE  
  
        ADD Swap(i, j) to moves  
  
    RETURN moves
```

6.4. 2-opt

Đảo ngược một đoạn trong cùng một trip:

$$[\dots, i_p, i_{p+1}, \dots, i_q, \dots] \rightarrow [\dots, i_q, i_{q-1}, \dots, i_p, \dots].$$

```
PROCEDURE GENERATE_2_OPT(solution s, selectedTrips):
```

```
    moves ← empty set
```

```
    FOR each trip t in selectedTrips:
```

```
        m ← size(t.customers)
```

```
        FOR p ← 0 TO m - 2:
```

```
            FOR q ← p + 1 TO m - 1:
```

```
                IF q == p + 1 and reversing produces no change:  
                    CONTINUE
```

```
                ADD TwoOpt(t, p, q) to moves
```

```
    RETURN moves
```

6.5. Cross-trip

Chọn hai trip và trao đổi hai đoạn đuôi.

Nếu:

$$\sigma_a = [A_1 \mid A_2], \quad \sigma_b = [B_1 \mid B_2],$$

thì sau move:

$$\sigma'_a = [A_1 \mid B_2], \quad \sigma'_b = [B_1 \mid A_2].$$

Hai trip có thể:

- Thuộc cùng phương tiện.
- Thuộc hai phương tiện khác nhau.
- Thuộc truck và drone, nếu các khách chuyển sang drone đều tương thích tĩnh.

```
PROCEDURE GENERATE_CROSS_TRIP(solution s, selectedTrips):
```



```
    moves ← empty set
```

```
    FOR each unordered pair of trips (ta, tb):
```

```
        FOR cutA ← 0 TO size(ta.customers):
```

```
            FOR cutB ← 0 TO size(tb.customers):
```

```
                tailA ← customers of ta from cutA onward
```

```
                tailB ← customers of tb from cutB onward
```

```
                IF tailA and tailB are both empty:
```

```
                    CONTINUE
```

```
                IF tailA contains a customer incompatible with vehicle of t
```

```
                    CONTINUE
```

```
                IF tailB contains a customer incompatible with vehicle of t
```

```
                    CONTINUE
```

```
                ADD CrossTrip(ta, tb, cutA, cutB) to moves
```

```
    RETURN moves
```

6.6. Trip-relocate

Di chuyển toàn bộ một trip:

- Sang vị trí khác trong chuỗi trip của cùng phương tiện.
- Sang một phương tiện tương thích khác.
- Sang bất kỳ vị trí nào trong chuỗi trip của phương tiện đích.


```
PROCEDURE GENERATE_TRIP_RELOCATE(solution s, selectedTrips):
```

```
    moves  $\leftarrow$  empty set
```

```
    FOR each trip t in selectedTrips:
```

```
        sourceVehicle  $\leftarrow$  vehicle containing t
```

```
        FOR each physical vehicle v:
```

```
            IF any customer of t is incompatible with v:  
                CONTINUE
```

```
            FOR destinationPosition  $\leftarrow$  0 TO numberOfTrips(v):
```

```
                IF destination is the original position:  
                    CONTINUE
```

```
                ADD TripRelocate(  
                    tripUid = t.uid,  
                    sourceVehicle,  
                    destinationVehicle = v,  
                    destinationPosition  
                )
```

```
    RETURN moves
```

7. Chọn candidate để giảm thời gian tính

Phiên bản đầu tiên nên sinh toàn bộ move để kiểm tra tính đúng đắn. Sau đó mới giới hạn candidate.

7.1. Khi chưa có nghiệm khả thi

Ưu tiên:

- Khách có đóng góp lớn vào V_{TW} .
- Khách có đóng góp lớn vào V_W .
- Khách thuộc trip vượt tải.
- Khách thuộc trip drone vượt tầm bay.
- Một tỷ lệ khách chọn ngẫu nhiên.

7.2. Khi đã có nghiệm khả thi

Xác định tập phương tiện critical:

$$\mathcal{V}^{\text{crit}} = \{v : C_v(s) = C_{\max}(s)\}.$$

Ưu tiên khách và trip thuộc các phương tiện này. Tuy nhiên, vẫn nên lấy thêm một tỷ lệ nhỏ nguồn ngẫu nhiên trên toàn hệ thống.

Đích của move luôn được xét trên tất cả phương tiện và trip tương thích.

```

FUNCTION SELECT_SEARCH_COMPONENTS(solution s, bestFeasible):

    IF bestFeasible == null:

        rank customers by their contribution to violations

        selectedCustomers ← top violating customers
                           + random customers

        selectedTrips ← trips containing selectedCustomers

    ELSE:

        criticalVehicles ← vehicles whose completion time
                           equals current makespan

        selectedCustomers ← customers on criticalVehicles
                           + random customers

        selectedTrips ← trips on criticalVehicles
                       + random trips

    RETURN selectedCustomers, selectedTrips

```

8. Đánh giá một move

Một move phải được áp dụng tạm thời, sau đó tính lại lịch của các phương tiện bị ảnh hưởng.

FUNCTION EVALUATE_MOVE(solution s, move m, penaltyWeights λ):

```

sPrime ← deep copy of s
oldAttributes ← EXTRACT_TABU_ATTRIBUTES(s, m)
APPLY_MOVE(sPrime, m)
remove all empty trips from sPrime

IF sPrime violates a structural constraint:

```

RETURN INVALID

affectedVehicles $\leftarrow$ vehicles modified by **m**

FOR each affected vehicle **v**:

firstAffectedTrip $\leftarrow$ earliest changed trip of **v**

RECOMPUTE_VEHICLE(

sPrime,

v,

firstAffectedTrip

)

EVALUATE_SOLUTION(**sPrime**, λ)

IF **sPrime** exceeds maximum allowed infeasibility:

RETURN INVALID

newAttributes $\leftarrow$ **EXTRACT_TABU_ATTRIBUTES**(**sPrime**, **m**)

candidate.solution $\leftarrow$ **sPrime**

candidate.move $\leftarrow$ **m**

candidate.removedAttributes $\leftarrow$ **oldAttributes** - **newAttributes**

candidate.addedAttributes $\leftarrow$ **newAttributes** - **oldAttributes**

RETURN candidate

Các ràng buộc cấu trúc phải loại ngay:

- Khách xuất hiện 0 lần hoặc nhiều hơn 1 lần.
- C_1 nằm trên drone.
- Sử dụng phương tiện ngoài tập $K + D$.
- Trip không bắt đầu/kết thúc tại depot.
- Khách được giao cho loại phương tiện không tương thích tĩnh.

Không loại candidate chỉ vì nó vượt tải, vượt tầm bay, trễ TW hoặc vượt L_w . Những vi phạm đó được đánh giá bằng hàm phạt.

9. Danh sách Tabu

9.1. Thuộc tính cung

Mỗi cung được biểu diễn:

(vehicleId, fromNode, toNode)



Cần tính cả cung:

- Depot $\rightarrow$ khách đầu tiên.
- Giữa hai khách.
- Khách cuối cùng $\rightarrow$ depot.

Sau khi thực hiện một move, các cung vừa bị xóa sẽ bị cấm tái tạo trong một số vòng lặp.

Ví dụ, Relocate khách i khỏi:

$\dots, p, i, q, \dots$

sẽ xóa:

$(p, i), \quad (i, q)$

và tạo:

$(p, q).$

Move đảo ngược sẽ phải tái tạo (p, i) và (i, q) , vì vậy nó bị tabu.

9.2. Thuộc tính phân công

Nếu khách i chuyển từ phương tiện v_a sang v_b , lưu:

```
("ASSIGN", customerId = i, forbiddenVehicle = va)
```



Trong thời hạn tabu, khách i không được quay lại v_a , trừ khi thỏa aspiration.

9.3. Thuộc tính thứ tự trip

Với Trip-relocate, lưu:

```
("TRIP_RETURN",  
 tripUid,  
 sourceVehicleId,  
 sourcePredecessorTripUid)
```



Thuộc tính này ngăn đưa trip quay ngay về vị trí cũ.

9.4. Tabu tenure

$$\tau \sim U[\tau_0, 2\tau_0].$$

Có thể giữ $\tau_0 = 7$ trong Chương 3 làm giá trị chạy thử ban đầu:

```
tenure ← RANDOM_INTEGER(tau0, 2 × tau0)  
tabuUntil[attribute] ← currentIteration + tenure
```



9.5. Kiểm tra tabu

```
FUNCTION IS_TABU(candidate c, tabuUntil, iteration):  
  
  FOR each attribute a in c.addedAttributes:  
  
    IF tabuUntil[a] > iteration:  
      RETURN true  
  
  RETURN false
```



9.6. Ghi tabu sau khi thực hiện move

```
PROCEDURE REGISTER_TABU(  
    selectedCandidate,  
    tabuUntil,  
    iteration,  
    tau0  
):  
  
    tenure ← RANDOM_INTEGER(tau0, 2 × tau0)  
  
    FOR each removed attribute a  
        in selectedCandidate.removedAttributes:  
  
            tabuUntil[a] ← iteration + tenure
```

10. Aspiration criterion

Move tabu vẫn được phép thực hiện trong hai trường hợp.

10.1. Đã có nghiệm khả thi

Candidate khả thi và cải thiện makespan tốt nhất:

$$V_{\Sigma}(s') = 0$$

và:

$$C_{\max}(s') < C_{\max}(s^*).$$

10.2. Chưa có nghiệm khả thi

Candidate cải thiện nghiệm không khả thi tốt nhất:

$$(V_{\Sigma}(s'), C_{\max}(s'), \text{Distance}(s')) < (V_{\Sigma}(s^I), C_{\max}(s^I), \text{Distance}(s^I)) .$$

Pseudocode:

```
FUNCTION SATISFIES_ASPIRATION(  
    candidate c,  
    bestFeasible,  
    bestInfeasible  
):  
  
    sPrime ← c.solution  
  
    IF IS_FEASIBLE(sPrime):  
  
        IF bestFeasible == null:  
            RETURN true  
  
        RETURN sPrime.makespan  
            < bestFeasible.makespan - epsilon  
  
    IF bestFeasible == null:  
        RETURN BETTER_INFEASIBLE(  
            sPrime,  
            bestInfeasible  
        )  
  
    RETURN false
```


Không sử dụng điều kiện:

$$F_{\lambda}(s') < F_{\lambda}(s^*)$$

vì hệ số λ thay đổi trong quá trình tìm kiếm.

11. Chọn move

11.1. Khi chưa tìm được nghiệm khả thi

Chọn move theo thứ tự từ điển:

$$(V_{\Sigma}(s'), \bar{C}_{\max}(s'), \text{Distance}(s')) .$$

Mục tiêu chính lúc này là đi về vùng khả thi.

11.2. Khi đã có nghiệm khả thi

Chọn theo:

$$(F_{\lambda}(s'), V_{\Sigma}(s'), \bar{C}_{\max}(s'), \text{Distance}(s')) .$$

Pseudocode:

```
FUNCTION SELECT_BEST_CANDIDATE(  
    candidatePool,  
    bestFeasible  
):  
  
    IF bestFeasible == null:  
  
        RETURN candidate minimizing:  
            (  
                candidate.solution.totalViolation,  
                candidate.solution.normalizedMakespan,  
                candidate.solution.totalDistance  
            )  
  
    RETURN candidate minimizing:  
        (  
            candidate.solution.penalizedObjective,  
            candidate.solution.totalViolation,  
            candidate.solution.normalizedMakespan,  
            candidate.solution.totalDistance  
        )
```



Move được chọn có thể xấu hơn nghiệm hiện tại. Đây là đặc trưng quan trọng của Tabu Search.



12. Strategic Oscillation

Mỗi ràng buộc có một hệ số phạt riêng:

$$\boldsymbol{\lambda} = (\lambda_Q, \lambda_D, \lambda_{TW}, \lambda_W).$$

Trong một segment gồm L_λ vòng lặp, đếm số nghiệm thỏa từng ràng buộc.

Với $g \in \{Q, D, TW, W\}$:

$$\rho_g = \frac{\#\{s_t : V_g(s_t) = 0\}}{L_\lambda}.$$

Cập nhật:

$$\lambda_g \leftarrow \begin{cases} \min\{\lambda_g^{\max}, \gamma_\uparrow \lambda_g\}, & \rho_g < \rho_{\min}, \\ \max\{\lambda_g^{\min}, \gamma_\downarrow \lambda_g\}, & \rho_g > \rho_{\max}, \\ \lambda_g, & \text{ngược lại.} \end{cases}$$

Pseudocode:

```
PROCEDURE UPDATE_PENALTIES(  
     $\lambda$ ,  
    feasibleCounts,  
    segmentLength  
):  
  
    FOR each constraint  $g$  in  $\{Q, D, TW, W\}$ :  
  
        ratio  $\leftarrow$  feasibleCounts[ $g$ ] / segmentLength  
  
        IF ratio < rhoMin:  
  
             $\lambda[g] \leftarrow \min(  
                \text{lambdaMax}[g],  
                \text{gammaIncrease} \times \lambda[g]  
            )$   
  
        ELSE IF ratio > rhoMax:  
  
             $\lambda[g] \leftarrow \max(  
                \text{lambdaMin}[g],  
                \text{gammaDecrease} \times \lambda[g]  
            )$   
  
        feasibleCounts[ $g$ ]  $\leftarrow$  0
```



Sau mỗi vòng lặp:

```
IF current.violationCapacity <= epsilon:  
    feasibleCounts[Q] += 1
```

```
IF current.violationRange <= epsilon:  
    feasibleCounts[D] += 1
```

```
IF current.violationTimeWindow <= epsilon:  
    feasibleCounts[TW] += 1
```

```
IF current.violationWaiting <= epsilon:  
    feasibleCounts[W] += 1
```

Đặt tạm các giá trị sau cho các tham số có trong thuật toán (sau này sẽ có experiment xác định giá trị cho chúng sau):

```
lambdaQ  = 1
lambdaD  = 1
lambdaTW = 1
lambdaW  = 1

lambdaMin = 0.001
lambdaMax = 1000

segmentLength = 20
rhoMin = 0.20
rhoMax = 0.80

gammaIncrease = 1.50
gammaDecrease = 0.85
```

13. Cập nhật nghiệm tốt nhất và stagnation

Sử dụng hai bộ đếm:

- `hDiv` : số vòng từ lần cải thiện hoặc diversification gần nhất.
- `hStop` : số vòng từ lần cải thiện record gần nhất.

FUNCTION UPDATE_BEST_SOLUTIONS(

current,

bestFeasible,

bestInfeasible

):

improvedRecord $\leftarrow$ false

IF IS_FEASIBLE(current):

IF bestFeasible == null

OR current.makespan

< bestFeasible.makespan - epsilon:

bestFeasible $\leftarrow$ deep copy of current

improvedRecord $\leftarrow$ true

ELSE:

IF BETTER_INFEASIBLE(

current,

bestInfeasible

):

bestInfeasible $\leftarrow$ deep copy of current

IF bestFeasible == null:

`improvedRecord  $\leftarrow$  true`

`RETURN bestFeasible,`

`bestInfeasible,`

`improvedRecord`

Cập nhật bộ đếm:

`IF improvedRecord:`

`hDiv  $\leftarrow$  0`

`hStop  $\leftarrow$  0`

`ELSE:`

`hDiv  $\leftarrow$  hDiv + 1`

`hStop  $\leftarrow$  hStop + 1`

14. Ruin & Recreate

Ruin & Recreate không phải move thông thường trong candidate pool. Nó chỉ được gọi khi:

$$h_{\text{Div}} \geq H_{\text{Div}}.$$

14.1. Ruin

Số khách bị loại:

$$q_{\text{ruin}} = \max\{1, \lfloor \rho_{\text{ruin}} n \rfloor\}.$$

Chọn hỗn hợp:

- Khoảng một phần ba khách có đóng góp vi phạm lớn.
- Khoảng một phần ba khách trên phương tiện critical.
- Phần còn lại chọn ngẫu nhiên.

Nếu nghiệm đang khả thi, tăng tỷ lệ lấy khách từ phương tiện critical.

14.2. Recreate

Các khách bị loại được:

1. Sắp theo l_i tăng dần.
2. Cho phép nhiều ngẫu nhiên nhẹ.
3. Chèn lại bằng best insertion trên toàn hệ thống.

Mỗi khách được thử tại:

- Mọi vị trí trong mọi trip tương thích.
- Trip mới tại mọi vị trí trong chuỗi trip của phương tiện tương thích.

Không ép khách vào `trucks[0]`.

14.3 Pseudocode

FUNCTION RUIN_RECREATE(

current,

penaltyWeights λ ,
bestFeasible,
ruinRate,
maximumAttempts
):

$q \leftarrow \max(1, \text{floor}(\text{ruinRate} \times n))$

FOR attempt $\leftarrow 1$ TO maximumAttempts:

 partial $\leftarrow$ deep copy of current

 removedCustomers $\leftarrow$ SELECT_RUIN_CUSTOMERS(
 partial,
 q
)

 REMOVE all removedCustomers from partial

 DELETE empty trips

 EVALUATE_SOLUTION(partial, λ)

 order removedCustomers by increasing latestTime

 apply small random perturbation to the order

success $\leftarrow$ true

FOR each customer i in removedCustomers:

insertionCandidates $\leftarrow$

 GENERATE_ALL_INSERTIONS(partial, i)

evaluatedInsertions $\leftarrow$ empty set

FOR each insertion move m

 in insertionCandidates:

 candidate $\leftarrow$ EVALUATE_MOVE(

 partial,

m ,

λ

)

 IF candidate is valid:

 ADD candidate to evaluatedInsertions

IF evaluatedInsertions is empty:

 success $\leftarrow$ false

BREAK

bestInsertion $\leftarrow$ **SELECT_BEST_CANDIDATE**(
 evaluatedInsertions,
 bestFeasible
)

partial $\leftarrow$ **bestInsertion.solution**

IF success:

RETURN partial

ruinRate $\leftarrow$ **max**(
 minimumRuinRate,
 $0.8 \times \text{ruinRate}$
)

RETURN FAILURE

15. Sinh candidate pool tổng hợp

FUNCTION BUILD_CANDIDATE_POOL(
 current,
 bestFeasible,
 penaltyWeights λ

);

candidatePool $\leftarrow$ empty set

selectedCustomers,

selectedTrips $\leftarrow$ SELECT_SEARCH_COMPONENTS(

current,

bestFeasible

)

rawMoves $\leftarrow$ empty set

rawMoves $\leftarrow$ rawMoves

U GENERATE_RELOCATE(

current,

selectedCustomers

)

rawMoves $\leftarrow$ rawMoves

U GENERATE_OR_OPT_2(

current,

selectedTrips

)

rawMoves $\leftarrow$ rawMoves

**U GENERATE_SWAP(
 current,
 selectedCustomers
)**

rawMoves $\leftarrow$ rawMoves

**U GENERATE_2_OPT(
 current,
 selectedTrips
)**

rawMoves $\leftarrow$ rawMoves

**U GENERATE_CROSS_TRIP(
 current,
 selectedTrips
)**

rawMoves $\leftarrow$ rawMoves

**U GENERATE_TRIP_RELOCATE(
 current,
 selectedTrips
)**

FOR each move m in rawMoves:

**candidate $\leftarrow$ EVALUATE_MOVE(
 current,**

m,
 λ
)

IF candidate is valid:

ADD candidate to candidatePool

RETURN candidatePool

16. Pseudo code tabu search

ALGORITHM ADAPTIVE_TABU_SEARCH

INPUT:

initial solution s0

maximum iterations Nmax

time limit Tlim

stopping stagnation HStop

diversification stagnation HDiv

base tabu tenure tau0

penalty segment length L λ

ruin rate pRuin

OUTPUT:

best feasible solution sBest

1. current $\leftarrow$ deep copy of s0

2. H $\leftarrow$ max(
1,

```

        makespan(s0),
        maximum customer deadline
    )
3.  $\lambda_Q \leftarrow 1$ 
    $\lambda_D \leftarrow 1$ 
    $\lambda_{TW} \leftarrow 1$ 
    $\lambda_W \leftarrow 1$ 
4. EVALUATE_SOLUTION(current,  $\lambda$ )

5. IF IS_FEASIBLE(current):
    bestFeasible  $\leftarrow$  deep copy of current
    bestInfeasible  $\leftarrow$  null
ELSE:
    bestFeasible  $\leftarrow$  null
    bestInfeasible  $\leftarrow$  deep copy of current
6. tabuUntil  $\leftarrow$  empty dictionary
7. feasibleCounts[Q]  $\leftarrow$  0
   feasibleCounts[D]  $\leftarrow$  0
   feasibleCounts[TW]  $\leftarrow$  0
   feasibleCounts[W]  $\leftarrow$  0
8. iteration  $\leftarrow$  0
   hDiv  $\leftarrow$  0
   hStop  $\leftarrow$  0
   startTime  $\leftarrow$  CURRENT_TIME()

```



```

9. WHILE iteration < Nmax
    AND CURRENT_TIME() - startTime < Tlim
    AND hStop < HStop:
10.  iteration ← iteration + 1
    // -----
    // Diversification
    // -----
11.  IF hDiv >= HDiv:
12.    repaired ← RUIN_RECREATE(
        current,
        λ,
        bestFeasible,
        pRuin,
        maximumAttempts
    )
13.    IF repaired != FAILURE:
14.        current ← repaired
        CLEAR tabuUntil
        hDiv ← 0
15.        bestFeasible,
        bestInfeasible,
        improved ← UPDATE_BEST_SOLUTIONS(
            current,
            bestFeasible,

```

```
        bestInfeasible  
    )
```

16. IF improved:

$hStop \leftarrow 0$

ELSE:

$hStop \leftarrow hStop + 1$

17. CONTINUE

```
// -----
```

```
// Generate and evaluate neighborhoods
```

```
// -----
```

18. $candidatePool \leftarrow BUILD_CANDIDATE_POOL($

 current,

 bestFeasible,

λ

)

19. $admissiblePool \leftarrow$ empty set

20. FOR each candidate c in candidatePool:

21. $tabu \leftarrow IS_TABU($

```

        c,
        tabuUntil,
        iteration
    )
22.    aspiration ← SATISFIES_ASPIRATION(
        c,
        bestFeasible,
        bestInfeasible
    )

23.    IF not tabu OR aspiration:
        ADD c to admissiblePool
        // -----
        // Avoid search termination caused by tabu
        // -----

24.    IF admissiblePool is empty:

25.        EXPAND candidate list and regenerate moves

26.    IF all candidates are still tabu:
        release the earliest-expiring tabu attribute
        and rebuild admissiblePool

27.    IF no valid move exists:
        hDiv ← HDiv
        CONTINUE

```

```

// -----
// Select and perform best admissible move
// -----
28.  selected  $\leftarrow$  SELECT_BEST_CANDIDATE(
        admissiblePool,
        bestFeasible
    )
29.  current  $\leftarrow$  selected.solution
30.  REGISTER_TABU(
        selected,
        tabuUntil,
        iteration,
        tau0
    )
// -----
// Update best solutions
// -----
31.  bestFeasible,
    bestInfeasible,
    improved  $\leftarrow$  UPDATE_BEST_SOLUTIONS(
        current,
        bestFeasible,
        bestInfeasible
    )

```

32. IF improved:

hDiv $\leftarrow$ 0

hStop $\leftarrow$ 0

ELSE:

hDiv $\leftarrow$ hDiv + 1

hStop $\leftarrow$ hStop + 1

// -----

// Collect feasibility statistics

// -----

33. IF current.violationCapacity $\leq$ epsilon:

feasibleCounts[Q] += 1

34. IF current.violationRange $\leq$ epsilon:

feasibleCounts[D] += 1

35. IF current.violationTimeWindow $\leq$ epsilon:

feasibleCounts[TW] += 1

36. IF current.violationWaiting $\leq$ epsilon:

feasibleCounts[W] += 1

// -----

// Strategic Oscillation

// -----

37. IF iteration mod $L\lambda$ == 0:

38. UPDATE_PENALTIES(

λ ,

```
feasibleCounts,  
    Lλ  
)
```

39. IF bestFeasible != null:

RETURN bestFeasible

40. ELSE:

REPORT "No feasible solution found"

RETURN bestInfeasible as diagnostic only

Thứ tự code cần hoàn thành:

1. Biểu diễn `Customer`, `Trip`, `Vehicle`, `Solution`.
2. `RECOMPUTE_VEHICLE`.
3. `EVALUATE_SOLUTION`.
4. Hàm kiểm tra nghiệm cuối cùng.
5. Relocate.
6. Swap.
7. 2-opt.
8. Or-opt(2).
9. Cross-trip.
10. Trip-relocate.
11. Tabu memory.
12. Aspiration.
13. Hàm lựa chọn move.
14. Strategic Oscillation.
15. Ruin & Recreate.
16. Candidate list và incremental evaluation.

Ở phiên bản đầu tiên, nên `deep copy` và tính lại toàn bộ nghiệm sau mỗi move. Khi tất cả unit test đúng mới tối ưu bằng cách chỉ tính lại các phương tiện và trip bị ảnh hưởng.